W06 — LLM Usage Dashboard (E-008 Screen 3)

Status at plan time: FE component + route + nav entry + backend endpoint already exist and are wired. The status report's "0%" is stale; real gaps are narrower than it implies. This plan closes those gaps.

Refs: PRD NF-U036, NF-037, US-A045. E-008 Screen 3.

---

1. Audit — what already ships today

Item	Where	Status
FE component `UsageDashboard.tsx`	`src/components/UsageDashboard/UsageDashboard.tsx` (1123 lines)	DONE
FE CSS module `UsageDashboard.module.css`	`src/components/UsageDashboard/UsageDashboard.module.css`	DONE
Summary KPI cards (tokens / requests / cost / latency)	lines 857–879 of the component	DONE
Stacked-area chart (inline SVG, no recharts dep)	lines 277–413	DONE
Filters (providers multi, project, from/to)	lines 748–840	DONE
By-Provider / By-Project / By-Model / By-User tabs	lines 926–1088	DONE
CSV export	lines 636–691	DONE
Role gating (ADMIN sees owner attribution section)	lines 1090–1118	DONE
Route `/analytics/llm-usage`	`src/App.tsx:209, 604–613`	DONE
Nav menu entry (all roles)	`src/App.tsx:336`	DONE
Backend `GET /v1/analytics/llm-usage`	`backend/src/routes/analytics.ts:89–403`	DONE
Backend rollups (`/providers`, `/projects`, `/cost-trend`)	`backend/src/routes/analytics.ts:467–672`	DONE
RLS-respecting SQL (ADMIN / PM / ANN+QA branches)	same file	DONE

Real gaps (what this plan delivers):

1. byDay aggregation is **total-per-day**, not per-day-per-provider. FE synthesises provider shares proportionally from period totals — stacked-area is therefore visually misleading (every day gets the same provider mix). Fix in SQL.
2. **No caching layer.** NF-U036 allows ≤5 min staleness; currently every request re-runs four GROUP BYs. Add a process-local TTL cache keyed on `(userId, role, from, to, projectId, provider)`.
3. **No vitest** for the analytics aggregate SQL. Plan file adds `backend/test/analytics-usage.test.ts`.
4. `groupBy=day|project|provider` query param is documented in the plan brief but not honoured by the endpoint (it always returns all four groupings). Keep the current "return everything" shape (FE consumes it all at once) but accept and ignore `groupBy` with a 400 on unknown values, so future partial responses can land without a FE breaking change.
5. `recharts` check — **not needed**. FE already uses inline SVG; do not add a new dep. Bundle impact of recharts (~95 KB gzipped + d3) would be unjustified.

---

2. Recharts decision — do not add

Check `package.json` (lines 20–30): deps are `@supabase/supabase-js`, `tailwindcss`, `framer-motion`, `lucide-react`, `papaparse`, `react`, `react-dom`, `xlsx`. No `recharts`.

Adding recharts would: - add ~95 KB gzipped (recharts ~55 KB + d3-scale/d3-shape ~40 KB) - require a tree-shake-friendly import style we don't use elsewhere - duplicate capability we already have (the inline `StackedArea` component handles N-series stacking, axis labels, empty state, and data-testid hooks)

Recommendation: keep inline SVG. If a second chart type is needed later (pie / sparkline / bar), add a small `src/components/charts/` utility rather than pulling in recharts.

---

3. Backend changes

3.1 Add per-day-per-provider aggregate (new response field)

File: `backend/src/routes/analytics.ts`

Add a fifth query (`byDayProviderRows`) running alongside the existing four, with the same three role branches. Shape:

```
interface ByDayProviderRow {
  day: string;            // YYYY-MM-DD
  provider_name: string;
  runs: string;
  tokens: string;
  cost_cents: string;
}
```

SQL body (ADMIN branch shown; PM and ANN/QA branches reuse the same body with the existing `scopeJoin`-style restriction — copy from the `byDayRows` pattern lines 194–209 / 269–285 / 345–361):

```
SELECT
  DATE(lr.created_at AT TIME ZONE 'UTC')::text                              AS day,
  COALESCE(
    lr.provider_snapshot->>'name',
    lr.provider_snapshot->>'provider_type',
    'unknown'
  )                                                                         AS provider_name,
  COUNT(*)::text                                                            AS runs,
  COALESCE(SUM(COALESCE(lr.tokens_prompt,0) + COALESCE(lr.tokens_completion,0)), 0)::text
                                                                            AS tokens,
  COALESCE(SUM(COALESCE(lr.cost_cents, 0)), 0)::text                        AS cost_cents
FROM public.llm_runs lr
JOIN public.claims c ON c.id = lr.claim_id
WHERE lr.created_at >= ${fromIso}::timestamptz
  AND lr.created_at <  ${toIso}::timestamptz
  ${project_id ? sql`AND c.project_id = ${project_id}::uuid` : sql``}
  ${provider   ? sql`AND (lr.provider_snapshot->>'name' ILIKE ${provider} OR lr.provider_snapshot->>'provider_type' ILI
GROUP BY 1, 2
ORDER BY 1 ASC, 2 ASC
```

Extend the response:

```
byDayProvider: byDayProviderRows.map((r) => ({
  date: r.day,
  provider: r.provider_name,
  runs: Math.round(n(r.runs)),
  tokens: Math.round(n(r.tokens)),
  costUsd: parseFloat((n(r.cost_cents) / 100).toFixed(4)),
})),
```

Keep `byDay` in the payload for FE back-compat; FE can prefer `byDayProvider` when present.

3.2 In-memory TTL cache (5-minute staleness, NF-U036)

New file: `backend/src/helpers/ttl-cache.ts` — minimal `Map`-backed cache. No redis (production is single-instance on Railway per CLAUDE.md; if we horizontally scale later, swap this module).

```
// backend/src/helpers/ttl-cache.ts
interface Entry<V> { value: V; expiresAt: number; }

export class TtlCache<V> {
  private store = new Map<string, Entry<V>>();
  constructor(private ttlMs: number, private maxEntries = 500) {}
```

```
  get(key: string): V | undefined {
    const hit = this.store.get(key);
    if (!hit) return undefined;
    if (hit.expiresAt < Date.now()) { this.store.delete(key); return undefined; }
    return hit.value;
  }

  set(key: string, value: V): void {
    if (this.store.size >= this.maxEntries) {
      // drop oldest (insertion order, Map iteration)
      const first = this.store.keys().next().value;
      if (first !== undefined) this.store.delete(first);
    }
    this.store.set(key, { value, expiresAt: Date.now() + this.ttlMs });
  }

  clear(): void { this.store.clear(); }
  size(): number { return this.store.size; }
}
```

In `analytics.ts`, top of module:

```
import { TtlCache } from '../helpers/ttl-cache.js';
const usageCache = new TtlCache<object>(5 * 60 * 1000); // 5 min per NF-U036
```

Inside the `/v1/analytics/llm-usage` handler, after parsing params and before running SQL:

```
const cacheKey = JSON.stringify({
  u: user.id, r: role,
  f: fromIso, t: toIso,
  p: project_id ?? null,
  v: provider ?? null,
});
const cached = usageCache.get(cacheKey);
if (cached) {
  reply.header('X-Cache', 'HIT');
  return reply.send(cached);
}
// ...run SQL + build payload...
usageCache.set(cacheKey, payload);
reply.header('X-Cache', 'MISS');
return reply.send(payload);
```

Cache key **must include `user.id` and `role`** — role-scoped queries return different rows per user. Missing this key would leak PM-only data to other PMs.

3.3 Accept-and-validate `groupBy`

Add to `LlmUsageQuerySchema`:

```
groupBy: z.enum(['day', 'project', 'provider']).optional(),
```

Currently ignored (response already includes all four groupings). Documented as "reserved for future trimmed responses" in the route comment block.

3.4 Unified diff — `backend/src/routes/analytics.ts`

```
@@ -24,6 +24,7 @@ import fp from 'fastify-plugin';
 import type { FastifyInstance, FastifyRequest, FastifyReply } from 'fastify';
 import { z } from 'zod';
 import { sql } from '../db.js';
 import { requireAuth } from '../plugins/rbac.js';
+import { TtlCache } from '../helpers/ttl-cache.js';

+// 5-minute TTL per NF-U036 (≤5 min staleness acceptable).
+const usageCache = new TtlCache<object>(5 * 60 * 1000);
+
```

```
 // ---------------------------------------------------------------------------
 // Query param schema
 // ---------------------------------------------------------------------------

 const LlmUsageQuerySchema = z.object({
   project_id: z.string().uuid().optional(),
   provider: z.string().min(1).max(80).optional(),
   from: z.string().optional(),
   to: z.string().optional(),
+  groupBy: z.enum(['day', 'project', 'provider']).optional(),
 });

@@ -66,6 +67,13 @@ interface ByDayRow {
   cost_cents: string;
 }

+interface ByDayProviderRow {
+  day: string;
+  provider_name: string;
+  runs: string;
+  tokens: string;
+  cost_cents: string;
+}
+
@@ -131,6 +139,16 @@ async function analyticsRoutes(fastify: FastifyInstance): Promise<void> {
       const fromIso = fromDate.toISOString();
       const toIso = toDate.toISOString();

+      // --- Cache lookup (NF-U036: ≤5 min staleness) ---
+      const cacheKey = JSON.stringify({
+        u: user.id, r: role,
+        f: fromIso, t: toIso,
+        p: project_id ?? null, v: provider ?? null,
+      });
+      const cached = usageCache.get(cacheKey);
+      if (cached) {
+        reply.header('X-Cache', 'HIT');
+        return reply.send(cached);
+      }
+
       // Build role-scoped CTE...

       let summaryRows: SummaryRow[];
       let byProviderRows: ByProviderRow[];
       let byProjectRows: ByProjectRow[];
       let byDayRows: ByDayRow[];
+      let byDayProviderRows: ByDayProviderRow[];
```

(Then inside each of the three role branches — ADMIN, PM, ANN/QA — append a fifth `await sql<ByDayProviderRow[]>\`...\`query using the SQL body from §3.1. The diff's body portion is mechanical — ~45 lines repeated three times. IncludeGROUP BY 1, 2`.)

Response object — extend with `byDayProvider` and cache-store before sending:

```
-      return reply.send({
+      const payload = {
         summary: { /* ... */ },
         byProvider: /* ... */,
         byProject: /* ... */,
         byDay: /* ... */,
+        byDayProvider: byDayProviderRows.map((r) => ({
+          date: r.day,
+          provider: r.provider_name,
+          runs: Math.round(n(r.runs)),
+          tokens: Math.round(n(r.tokens)),
+          costUsd: parseFloat((n(r.cost_cents) / 100).toFixed(4)),
+        })),
-      });
```

```
+      };
+      usageCache.set(cacheKey, payload);
+      reply.header('X-Cache', 'MISS');
+      return reply.send(payload);
```

---

4. FE: prefer `byDayProvider` for the stacked chart

File: `src/components/UsageDashboard/UsageDashboard.tsx`

Currently the chart synthesises per-provider daily values by applying the period-wide provider share to each day's total (lines 584–624). Replace the `stackedPoints` `useMemo` to prefer real per-day-per-provider data when the backend sends it, falling back to the current synthesis for back-compat.

Add to types (after `ByDayRow`, ~line 117):

```
interface ByDayProviderRow {
  date?: string;
  day?: string;
  provider?: string;
  name?: string;
  runs?: number;
  tokens?: number;
  costUsd?: number;
  cost_usd?: number;
}
```

Add to `UsageResponse`:

```
byDayProvider?: ByDayProviderRow[];
by_day_provider?: ByDayProviderRow[];
```

Add a derived memo:

```
const byDayProvider = useMemo<ByDayProviderRow[]>(
  () => data?.byDayProvider ?? data?.by_day_provider ?? [],
  [data],
);
```

Replace the `stackedPoints` memo body:

```
const stackedPoints = useMemo<StackedPoint[]>(() => {
  // Preferred path: real per-day-per-provider rows from backend.
  if (byDayProvider.length > 0) {
    const byDate = new Map<string, Record<string, number>>();
    const displayed = providerFilter.length > 0
      ? new Set(providerFilter)
      : null;
    for (const r of byDayProvider) {
      const d = r.date ?? r.day ?? '';
      if (!d) continue;
      const prov = r.provider ?? r.name ?? 'unknown';
      if (displayed && !displayed.has(prov)) continue;
      const v = chartMetric === 'tokens'
        ? (r.tokens ?? 0)
        : (r.costUsd ?? r.cost_usd ?? 0);
      const bucket = byDate.get(d) ?? {};
      bucket[prov] = (bucket[prov] ?? 0) + v;
      byDate.set(d, bucket);
    }
    return [...byDate.entries()]
      .sort(([a], [b]) => a.localeCompare(b))
      .map(([date, values]) => ({ date, values }));
  }
  // Fallback: synthesise from byDay totals + period-wide provider shares.
  // (existing body — lines 585–623 — retained verbatim)
  if (byDay.length === 0) return [];
```

```
  /* ...existing synthesis... */
}, [byDayProvider, byDay, byProvider, allProviders, providerFilter, chartMetric]);
```

No CSS changes. No new data-testid values.

---

5. New file — `backend/src/helpers/ttl-cache.ts`

Full contents in §3.2 above. Parent directory confirmed to exist (`backend/src/helpers/`).

---

6. New test — `backend/test/analytics-usage.test.ts`

Model after the shape of `backend/test/routes/auth.test.ts` (build Fastify app with embedded-postgres, call route via `app.inject`).

```
// backend/test/analytics-usage.test.ts
import { describe, it, expect, beforeAll, afterAll } from 'vitest';
import { buildApp } from '../src/app.js';         // (or however tests build the app)
import { sql } from '../src/db.js';
import type { FastifyInstance } from 'fastify';

// Helpers: create_user, login → returns { token, userId }. Reuse existing
// fixtures from test/routes/auth.test.ts or test-helpers.

describe('GET /v1/analytics/llm-usage', () => {
  let app: FastifyInstance;
  let adminToken: string;
  let pmToken: string;
  let pmId: string;
  let projectId: string;

  beforeAll(async () => {
    app = await buildApp({ logger: false });
    // seed: 1 ADMIN, 1 PM member of 1 project, 1 project NOT member, 1 other project
    //       10 llm_runs on member project (varied providers, dates in last 7 days)
    //       5  llm_runs on non-member project
    // TODO: extract seed helper into test-helpers.ts
  });

  afterAll(async () => { await app.close(); });

  it('ADMIN sees all llm_runs across workspace', async () => {
    const res = await app.inject({
      method: 'GET',
      url: '/v1/analytics/llm-usage?from=2026-04-01&to=2026-04-30',
      headers: { authorization: `Bearer ${adminToken}` },
    });
    expect(res.statusCode).toBe(200);
    const body = res.json();
    expect(body.summary.totalRuns).toBe(15);  // 10 + 5
    expect(body.byProject).toHaveLength(2);
  });

  it('PM sees only llm_runs for projects they are a member of', async () => {
    const res = await app.inject({
      method: 'GET',
      url: '/v1/analytics/llm-usage?from=2026-04-01&to=2026-04-30',
      headers: { authorization: `Bearer ${pmToken}` },
    });
    expect(res.statusCode).toBe(200);
    const body = res.json();
    expect(body.summary.totalRuns).toBe(10);
    expect(body.byProject).toHaveLength(1);
```

```
    expect(body.byProject[0].projectId).toBe(projectId);
  });

  it('returns byDayProvider grouped by (day, provider)', async () => {
    const res = await app.inject({
      method: 'GET',
      url: '/v1/analytics/llm-usage?from=2026-04-01&to=2026-04-30',
      headers: { authorization: `Bearer ${adminToken}` },
    });
    const body = res.json();
    expect(Array.isArray(body.byDayProvider)).toBe(true);
    // At least one row per (day, provider) combination present in fixture.
    const keys = new Set(body.byDayProvider.map((r: {date:string;provider:string}) =>
      `${r.date}|${r.provider}`));
    expect(keys.size).toBeGreaterThanOrEqual(2);
  });

  it('filters by project_id', async () => {
    const res = await app.inject({
      method: 'GET',
      url: `/v1/analytics/llm-usage?from=2026-04-01&to=2026-04-30&project_id=${projectId}`,
      headers: { authorization: `Bearer ${adminToken}` },
    });
    const body = res.json();
    expect(body.byProject).toHaveLength(1);
    expect(body.byProject[0].projectId).toBe(projectId);
  });

  it('rejects unknown groupBy values', async () => {
    const res = await app.inject({
      method: 'GET',
      url: '/v1/analytics/llm-usage?groupBy=bogus',
      headers: { authorization: `Bearer ${adminToken}` },
    });
    expect(res.statusCode).toBe(400);
  });

  it('serves from cache on the second identical request (X-Cache: HIT)', async () => {
    const url = '/v1/analytics/llm-usage?from=2026-04-01&to=2026-04-30';
    const headers = { authorization: `Bearer ${adminToken}` };
    const r1 = await app.inject({ method: 'GET', url, headers });
    expect(r1.headers['x-cache']).toBe('MISS');
    const r2 = await app.inject({ method: 'GET', url, headers });
    expect(r2.headers['x-cache']).toBe('HIT');
    expect(r2.json()).toEqual(r1.json());
  });

  it('cache key is scoped by user — PM does not see ADMIN cached rows', async () => {
    const url = '/v1/analytics/llm-usage?from=2026-04-01&to=2026-04-30';
    await app.inject({ method: 'GET', url,
      headers: { authorization: `Bearer ${adminToken}` } });
    const pmRes = await app.inject({ method: 'GET', url,
      headers: { authorization: `Bearer ${pmToken}` } });
    expect(pmRes.headers['x-cache']).toBe('MISS');
    expect(pmRes.json().summary.totalRuns).toBe(10);
  });
});
```

Run with `cd backend && npm run test -- analytics-usage`.

---

7. Nav wiring — already present, just verify

- `src/App.tsx:24` — import

- `src/App.tsx:209` — SPA route recognised
- `src/App.tsx:336` — menu entry for ADMIN/PM/QA/ANN
- `src/App.tsx:604–613` — render path

No diff required. Double-check the menu entry's `roles` array still matches the PRD (ADMIN sees all, PM sees own; QA/ANN may view their own runs — current `ANN/QA` backend branch restricts by assignment, which matches US-A045).

---

8. Risks / open questions

- **Backend test harness pattern**: existing tests under `backend/test/routes/` may already export a `buildApp` / seed helper. Before writing `analytics-usage.test.ts`, read `backend/test/routes/reviews.test.ts` to confirm the seed pattern (likely `sql` inserts in `beforeAll`).
- **Cache invalidation on writes**: new `llm_runs` rows arrive from the judge worker, not from HTTP handlers we control. 5-min staleness is acceptable per NF-U036, so no active invalidation. Document in the route comment.
- **Horizontal scaling**: `TtlCache` is per-process. If Railway scales the backend to >1 instance, cache hit rate drops but correctness is unchanged. Swap to redis when/if that happens — interface-compatible.
- **Existing `byDay` proportional synthesis** will still ship in the FE as the fallback path. Remove after one release cycle when all clients have the new backend.

---

9. Acceptance checklist

- [ ] `backend/src/helpers/ttl-cache.ts` added
- [ ] `backend/src/routes/analytics.ts` patched (cache + `byDayProvider` + `groupBy` schema)
- [ ] `src/components/UsageDashboard/UsageDashboard.tsx` prefers `byDayProvider` when present
- [ ] `backend/test/analytics-usage.test.ts` passes (7 cases above)
- [ ] `npm run lint` clean (frontend)
- [ ] `cd backend && npm run test` clean
- [ ] Manual smoke: open `/analytics/llm-usage` as ADMIN + PM, confirm chart, tabs, filters, CSV export, `X-Cache: MISS→HIT` in devtools
- [ ] Verify `X-Cache: HIT` response is <50 ms p95 (versus cold ~200–500 ms)